



B-Tree ve B+Tree

Ömer Faruk Zil 02240224124
Veri Org.



B-Tree

Her düğüm hem anahtar hem de veri tutabilir. Arama, herhangi bir düğümde sonuçlanabilir.

- ✓ Her düğümde veri saklanır
- ✓ Rastgele erişimde verimli
- ⚠ Sıralı tarama yavaştır
- ⚠ Daha fazla bellek kullanır

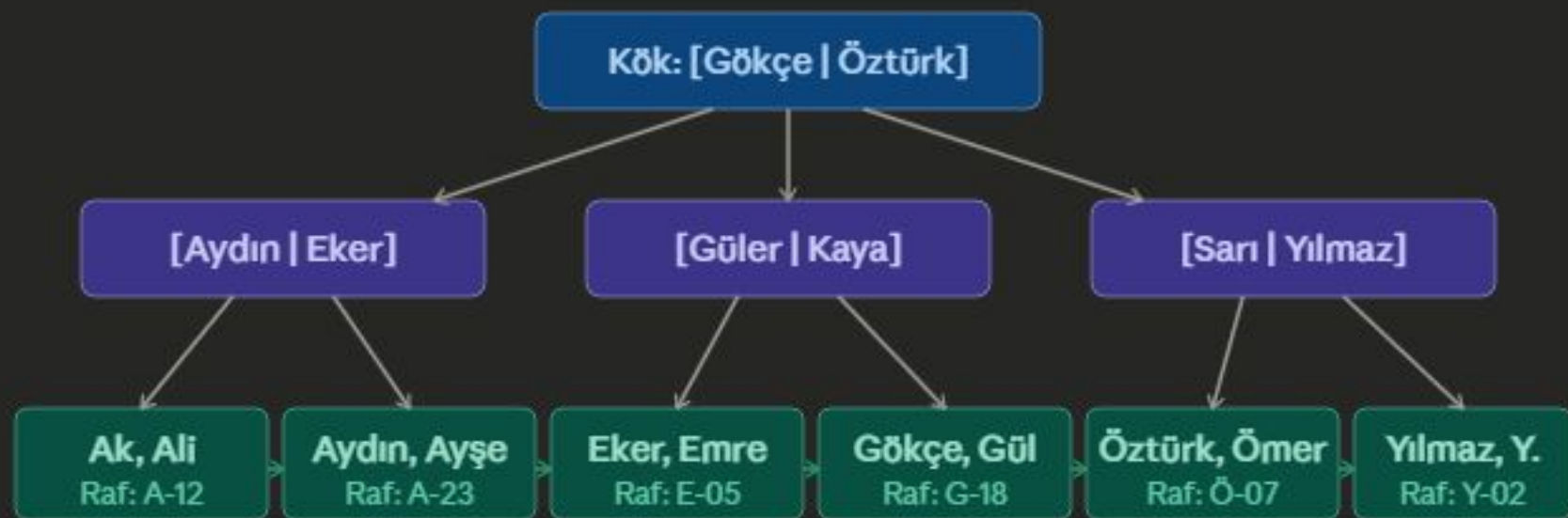


B+ Tree

Sadece yaprak düğümlerde veri bulunur. Yapraklar bağlı liste gibi birbirine zincirlenmiştir.

- ✓ Yapraklar bağlı liste → hızlı sıralı tarama
- ✓ İç düğümler yalnızca anahtar → daha fazla anahtar sığar
- ✓ Disk I/O sayısı azalır
- ✓ Aralık sorguları çok hızlı

2 B+ Tree Yapısı (Derecesi 3, Kitaplık Örneği)



■ Yaprak düğümler bağlı liste oluşturur → sıralı tarama $O(n)$



Veritabanlarında Kullanım

MySQL

İlişkisel Veritabanı

InnoDB depolama motoru, hem birincil hem de ikincil indeksleri **B+ Tree** ile saklar. Birincil anahtar, satır verilerini doğrudan yaprak düğümlerde barındırır (Clustered Index). Milyonlarca satırı milisaniyeler içinde sorgular.

PostgreSQL

İlişkisel Veritabanı

Varsayılan indeks türü **B-Tree**'dir. WHERE, ORDER BY, GROUP BY, BETWEEN gibi sorguları optimize eder. Ayrıca Hash, GIN, GiST, BRIN gibi özel indeks türleri de sunar; ancak genel kullanım için B-Tree seçilir.

MongoDB

Belge Tabanlı NoSQL

WiredTiger depolama motoru **B-Tree** tabanlı indeksleme kullanır. JSON benzeri BSON belgelerini indeksleyerek karmaşık sorgularda yüksek performans sağlar. Bileşik ve çok anahtar indeksleri destekler.



Dosya Sistemleri

NTFS (Windows): MFT (Master File Table) B+ Tree yapısında dizinleri depolar.

ext4 (Linux): HTree (B-Tree türevi) ile büyük dizinlerde dosya aramasını $O(\log n)$ 'e indirger.

HFS+ ve **APFS** (macOS): B-Tree tabanlı meta veri yönetimi kullanır.

Buyuk Veri & Arama

Apache HBase: LSM-Tree ile B-Tree'yi birleştiren hibrit yapı kullanır.

Google Bigtable: Sıralı anahtar yapısı B-Tree mantığına dayanır.

Elasticsearch: Lucene tabanlı; inverted index ile B-Tree indekslerini birlikte kullanır.

SQLite: Gömülü sistemlerde saf B-Tree uygulaması.



Gunluk Hayattan Benzetme

Telefon Rehberi

Eski büyük şehir telefon rehberlerini düşünün. B+ Tree tam olarak bu yapıyı simüle eder.

Kök: Rehberin kapağındaki büyük bölümler (A-F, G-M, N-Z).

İç düğümler: Her bölümün başındaki alt grup rehberleri (Ak-, Al-, Am-...).

Yaprak düğümler: Gerçek isim ve telefon numaraları. Her sayfa bir sonraki sayfayla bağlantılıdır, "Kaya'dan Kırmızı'ya kadar tüm aboneler" demek yalnızca o sayfaları okumak demektir — kitabın başına dönmek gerekmez.

Pratik faydası: "042 ile başlayan tüm numaralar" gibi bir sorgu, sayfalara gidip oradan ilerleyerek $O(k)$ adımda tamamlanır; k , bulunan sonuç sayısıdır.

Neden Önemli ? Neden Hala Kullanılıyor?



$O(\log n)$ Garanti

1 milyar kayıt içinde arama yaparken bile en fazla 30 adım yeterlidir. Bu matematiksel güvence, sistem tasarımcılarının performansı öngörülmesini sağlar.



Disk I/O Verimliliği

Her düğüm bir disk bloğuna (genellikle 4KB-16KB) sığacak şekilde tasarlanır. Bu, disk okuma sayısını minimize eder — RAM'den binlerce kat yavaş olan diske erişim sayısını azaltmak kritiktir.



Aralık Sorguları

SQL'deki BETWEEN, ORDER BY, LIMIT sorguları B+ Tree'nin bağlı yaprak listesi sayesinde mükemmel çalışır. E-ticaret, finans, sağlık gibi sektörlerin en yaygın sorguları bunlardır.



Kendi Kendine Dengeleme

Veri ekleme/silme işlemlerinde ağaç otomatik olarak yeniden düzenlenir. AVL veya Red-Black Tree gibi basit yapıların aksine, büyük veri hacimleriyle başa çıkacak şekilde tasarlanmıştır.



Kisisel Yorumlar

Bu veri yapıları neden önemlidir?

B ve B+ ağaçları, bilgisayar biliminin en başarılı soyutlamalarından biridir. Bir kullanıcı Google'da arama yaptığında, bankasına para transferi gönderdiğinde ya da e-ticaret sitesinde ürün filtresi uyguladığında farkında olmadan bu yapılarla etkileşime girer. Önemleri sadece teknik değil, toplumsal düzeydedir: hızlı ve güvenilir yazılım altyapısının temelini oluştururlar.

Büyük veri çaığında neden ihtiyaç duyulmaktadır?

2024 itibarıyla dünyada her gün 2,5 kentilyon bayt veri üretilmektedir. Doğrusal arama ile bu veriyi işlemek mümkün değildir. B+ Tree, veri büyüklüğü arttıkça performansın logaritmik olarak ölçeklenmesini sağlar. Yatay ölçeklenme (sharding) ile birleştirildiğinde petabayt ölçeğindeki sistemlerde bile geçerliliğini korur. Bunun yanı sıra NVMe SSD'lerin yaygınlaşması ile ağacın disk bloğu hizalaması avantajı daha da artmıştır.

Gelecekte de kullanılmaya devam edecek midir?

Büyük olasılıkla evet — ancak yeni rakipler de mevcuttur. LSM-Tree (Log-Structured Merge Tree) yazma yoğun iş yüklerinde (Cassandra, RocksDB) B-Tree'nin önüne geçmektedir. Persistent Memory (Optane gibi) teknolojileriyle birlikte ağaç yapıları değişime uğrayacaktır. Bununla birlikte, onlarca yıllık optimizasyon geçmiş, geniş ekosistem desteği ve okunabilirlik yoğun iş yüklerindeki üstünlüğü göz önüne alındığında B+ Tree'nin önümüzdeki on yıllarda da kritik sistemlerde yer almaya devam edeceğini düşünüyorum.